

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ЛАБОРАТОРНАЯ РАБОТА № 2

По дисциплине: «Frontend разработка»

Тема: «Взаимодействие с внешним API»

Вариант: «Платформа для онлайн-курсов и обучения»

Выполнил:
студент группы **J3210**
Райков Илья Алексеевич

Содержание

1	Цель и задачи работы	2
2	Архитектура проекта	2
2.1	Файловая структура	2
3	Описание реализации	2
3.1	Настройка Mock API (Backend)	2
3.2	Асинхронное взаимодействие (Frontend)	3
4	Листинги основного кода	3
4.1	Асинхронная авторизация и получение токена (auth.js)	3
4.2	Получение списка курсов с сервера (filters.js)	4
4.3	Защищенный PATCH-запрос с использованием Bearer токена (modal.js)	4
5	Заключение	5

1 Цель и задачи работы

Цель лабораторной работы: получить практические навыки реализации клиент-серверной архитектуры веб-приложения. Освоить механизмы асинхронного взаимодействия Frontend-части с внешним REST API средствами встроенного браузерного API `fetch`, а также реализовать систему авторизации на основе токенов (JWT).

Задачи:

- Развернуть мок-сервер (Mock API) с использованием библиотек `json-server` и `json-server-auth`.
- Спроектировать структуру базы данных в формате JSON (`db.json`).
- Заменить локальную логику хранения данных (`localStorage`) на выполнение асинхронных HTTP-запросов (GET, POST, PATCH).
- Реализовать механизмы регистрации и авторизации пользователей с получением и сохранением токена доступа (`accessToken`).
- Обеспечить передачу Bearer-токена в заголовках при выполнении защищенных запросов (покупка курса, изменение пароля, получение данных профиля).
- Реализовать обработку ошибок сети и статусов HTTP-ответов сервера.

2 Архитектура проекта

В рамках данной лабораторной работы проект был разделен на независимые Frontend и Backend части. Роль Backend выполняет локальный Node.js сервер.

2.1 Файловая структура

├── db.json	# База данных сервера (пользователи, курсы)
├── package.json	# Файл зависимостей npm и скриптов запуска
├── index.html	# Главная страница (Каталог и фильтры)
├── login.html	# Страница авторизации
├── register.html	# Страница регистрации
├── dashboard.html	# Личный кабинет пользователя
├── css/	
│ └── ...	# Файлы стилей (без изменений)
├── js/	
│ ├── main.js	# Точка входа приложения
│ ├── auth.js	# Асинхронная логика регистрации и авторизации
│ ├── filters.js	# Получение курсов с API (GET) и фильтрация
│ ├── modal.js	# Обновление профиля пользователя (PATCH)
│ ├── session.js	# Управление токенами и состоянием сессии
│ └── dashboard.js	# Загрузка актуальных данных профиля с API

3 Описание реализации

3.1 Настройка Mock API (Backend)

Для имитации полноценного REST API была использована библиотека `json-server` в связке с `json-server-auth`. В качестве хранилища данных выступает файл `db.json`,

содержащий коллекции `users` и `courses`. Библиотека `json-server-auth` берет на себя ответственность за хеширование паролей при регистрации и генерацию JWT-токенов при входе. Запуск сервера осуществляется командой `npm run server` на порту 3000.

3.2 Асинхронное взаимодействие (Frontend)

Вся бизнес-логика приложения была переписана с использованием современного стандарта `async/await` и функции `fetch`:

1. **Загрузка данных (GET):** При загрузке главной страницы (`filters.js`) Frontend отправляет GET-запрос на эндпоинт `/courses`. Полученный JSON-массив используется для рендеринга карточек курсов и последующей фильтрации.
2. **Авторизация (POST):** При отправке формы входа (`auth.js`) выполняется POST-запрос на эндпоинт `/login`. В случае успеха сервер возвращает объект пользователя и `accessToken`. Токен сохраняется в `localStorage` для поддержания сессии (Persistence).
3. **Защищенные маршруты (PATCH):** При покупке курса (`modal.js`) или смене пароля Frontend модифицирует данные текущего пользователя, отправляя PATCH-запрос на `/users/{id}`. Для прохождения авторизации в заголовки запроса добавляется `Authorization: Bearer <token>`.
4. **Обработка ошибок:** Во всех асинхронных функциях реализованы блоки `try...catch` для перехвата сетевых ошибок, а также проверка свойства `response.ok` для обработки HTTP-ошибок (например, 400 Bad Request при неверном логине/пароле).

4 Листинги основного кода

Ниже представлены выдержки из обновленных файлов, демонстрирующих работу с Fetch API и токенами.

4.1 Асинхронная авторизация и получение токена (auth.js)

```
1 loginForm.addEventListener("submit", async function (event) {
2   event.preventDefault();
3   loginBtn.innerHTML = '<span class="spinner-border spinner-border-sm"></span>';
4
5   try {
6     const response = await fetch('http://127.0.0.1:3000/login', {
7       method: 'POST',
8       headers: { 'Content-Type': 'application/json' },
9       body: JSON.stringify({
10        email: emailInput.value,
11        password: passwordInput.value
12      })
13    });
14
15    if (response.ok) {
16      const { accessToken, user } = await response.json();
17      localStorage.setItem("accessToken", accessToken);
18      localStorage.setItem("user", JSON.stringify(user));
19      window.location.href = "dashboard.html";
20    }
21  }
22 });
```

```

20     } else {
21         passwordInput.setCustomValidity("Неверный email или пароль")
22     };
23     loginForm.reportValidity();
24 } catch (error) {
25     alert("Ошибка сети");
26 }
27 });

```

4.2 Получение списка курсов с сервера (filters.js)

```

1 export async function initFilters() {
2     const coursesContainer = document.querySelector(".col-lg-9 .row.g-4"
3 );
4     let COURSES_DATA = [];
5     try {
6         const response = await fetch('http://127.0.0.1:3000/courses');
7         if (!response.ok) throw new Error(`Ошибка сервера: ${response.
8 status}`);
9         COURSES_DATA = await response.json();
10    } catch (error) {
11        coursesContainer.innerHTML = `
12            <div class="col-12 text-center text-danger py-5">
13                <h4Ошибка> загрузки курсов с сервера</h4>
14            </div>`;
15        return;
16    }
17    renderCourses(COURSES_DATA);
18    // ... логика локальной фильтрации ...
19 }

```

4.3 Защищенный PATCHN-запрос с использованием Bearer токена (modal.js)

```

1 confirmBtn.addEventListener("click", async function () {
2     const user = getCurrentUser();
3     const token = getAuthToken();
4
5     if (!user || !token) {
6         window.location.href = "login.html";
7         return;
8     }
9
10    try {
11        const userRes = await fetch(`http://127.0.0.1:3000/users/${user.
12 id}`, {
13            headers: { 'Authorization': `Bearer ${token}` }
14        });
15        const userData = await userRes.json();

```

```

16     userData.courses.push(currentSelectedCourse);
17
18     const updateRes = await fetch(`http://127.0.0.1:3000/users/${
19   user.id}`, {
20       method: 'PATCH',
21       headers: {
22         'Content-Type': 'application/json',
23         'Authorization': `Bearer ${token}`
24       },
25       body: JSON.stringify({ courses: userData.courses })
26     });
27
28     if (updateRes.ok) {
29       localStorage.setItem("user", JSON.stringify(await updateRes.
30   json()));
31       alert("Оплата прошла успешно!");
32       window.location.href = "dashboard.html";
33     } catch (error) {
34       alert("Ошибка соединения с сервером");
35     }
36   });

```

5 Заключение

В ходе выполнения второй лабораторной работы приложение было успешно переведено на клиент-серверную архитектуру. Был развернут локальный Mock-сервер с использованием `json-server`, спроектирована структура JSON-коллекций для курсов и пользователей.

Было освоено встроенное браузерное API `fetch` в связке с синтаксисом `async/await`, что позволило избавиться от блокирующих операций и сделать приложение по-настоящему реактивным. Реализована полноценная система безопасности: пароли пользователей хешируются на стороне сервера, а сессии управляются посредством JSON Web Tokens (JWT). Получены навыки добавления HTTP-заголовков (в частности, `Authorization: Bearer`) для защиты приватных маршрутов API.